

Vector

- 构造

```
vector<int> v;           // 空
vector<int> v(n);        // n个0
vector<int> v(n, x);     // n个x
vector<int> v = {1,2,3}; // 初始化列表
vector<int> v2(v);       // 拷贝
vector<int> v(arr, arr+n); // 从数组构造
```

- 增删

```
v.push_back(x);           // 末尾加
v.pop_back();             // 末尾删
v.insert(v.begin()+i, x); // 位置i插入单个
v.insert(v.begin()+i, n, x); // 位置i插入n个x
v.insert(v.begin()+i, v2.begin(), v2.end()); // 插入另一段
v.erase(v.begin()+i);     // 删除位置i
v.erase(v.begin()+l, v.begin()+r); // 删除[l,r)
v.clear();                // 清空
```

- 访问

```
v[i];           // 下标 (不检查越界)
v.at(i);        // 下标 (检查越界, 抛异常)
v.front();      // 第一个
v.back();       // 最后一个
v.data();       // 返回底层数组指针
```

- 大小

```
v.size();       // 元素个数
v.empty();      // 是否为空
v.resize(n);     // 改变大小, 新元素为0
v.resize(n, x);  // 改变大小, 新元素为x
v.reserve(n);    // 预分配内存 (不改变size)
v.capacity();    // 当前分配的内存容量
v.shrink_to_fit(); // 释放多余内存
```

- 遍历

```

for (int i = 0; i < v.size(); i++) v[i];
for (auto x : v) x;
for (auto& x : v) x; // 可修改
for (auto it = v.begin(); it != v.end(); it++) *it;

```

常用算法 (需 `<algorithm>`)

```

sort(v.begin(), v.end()); // 升序
sort(v.begin(), v.end(), greater<int>()); // 降序
sort(v.begin(), v.end(), cmp); // 自定义比较
reverse(v.begin(), v.end()); // 反转
find(v.begin(), v.end(), x); // 查找, 返回迭代器
count(v.begin(), v.end(), x); // 统计x出现次数
max_element(v.begin(), v.end()); // 返回最大值迭代器
min_element(v.begin(), v.end()); // 返回最小值迭代器
accumulate(v.begin(), v.end(), 0); // 求和 (需<numeric>)
unique(v.begin(), v.end()); // 去重 (需先排序)
lower_bound(v.begin(), v.end(), x); // 第一个>=x的迭代器
upper_bound(v.begin(), v.end(), x); // 第一个>x的迭代器

```

Set

• 构造

```

set<int> s; // 空
set<int> s = {1,2,3}; // 初始化列表
set<int> s(v.begin(), v.end()); // 从vector构造
set<int, greater<int>> s; // 降序

```

• 增删

```

s.insert(x); // 插入x (已存在则忽略)
s.erase(x); // 按值删除
s.erase(s.find(x)); // 按迭代器删除
s.erase(it1, it2); // 删除[it1,it2)
s.clear(); // 清空

```

• 查找

```

s.find(x); // 返回迭代器, 不存在返回s.end()
s.count(x); // 存在返回1, 不存在返回0
s.contains(x); // 是否存在 (C++20)
s.lower_bound(x); // 第一个>=x的迭代器

```

```
s.upper_bound(x);    // 第一个>x的迭代器
s.equal_range(x);    // 返回pair(lower_bound, upper_bound)
```

- 大小

```
s.size();            // 元素个数
s.empty();           // 是否为空
```

- 遍历 (有序)

```
for (auto x : s)      x;
for (auto it = s.begin(); it != s.end(); it++) *it;
for (auto it = s.rbegin(); it != s.rend(); it++) *it; // 反向
```

Multiset

与 set 基本相同，区别是允许重复元素

- 构造

```
multiset<int> ms;
multiset<int> ms = {1,1,2,3};
multiset<int, greater<int>> ms; // 降序
```

- 增删

```
ms.insert(x);          // 插入x (允许重复)
ms.erase(x);           // 删除所有值为x的元素
ms.erase(ms.find(x));  // 只删除一个x
ms.erase(it1, it2);    // 删除[it1,it2)
ms.clear();
```

- 查找

```
ms.find(x);            // 返回第一个x的迭代器
ms.count(x);           // x出现的次数
ms.contains(x);        // 是否存在 (C++20)
ms.lower_bound(x);     // 第一个>=x的迭代器
ms.upper_bound(x);     // 第一个>x的迭代器
ms.equal_range(x);     // 返回pair(lower_bound, upper_bound)
```

- 大小

```
ms.size();  
ms.empty();
```

- 遍历 (有序)

```
for (auto x : ms)    x;  
for (auto it = ms.rbegin(); it != ms.rend(); it++) *it; // 反向
```

Priority Queue

- 构造

```
priority_queue<int> pq; // 大根堆 (默认)  
priority_queue<int, vector<int>, greater<int>> pq; // 小根堆  
priority_queue<int> pq(v.begin(), v.end()); // 从vector构造  
// 自定义比较  
auto cmp = [](int a, int b){ return a > b; };  
priority_queue<int, vector<int>, decltype(cmp)> pq(cmp);
```

- 增删

```
pq.push(x); // 插入x  
pq.pop(); // 删除堆顶 (不返回值)  
pq.emplace(x); // 原地构造插入 (比push略快)
```

- 访问

```
pq.top(); // 查看堆顶 (大根堆为最大值)
```

- 大小

```
pq.size();  
pq.empty();
```

- 注意事项

```
// 不支持遍历、不支持find、不支持修改内部元素
// 取出所有元素只能循环pop
while (!pq.empty()) {
    cout << pq.top();
    pq.pop();
}

// pair 默认按 first 降序, first 相同按 second 降序
priority_queue<pair<int,int>> pq;
```

iota

定义在 `<numeric>` 里, 用来给区间填充连续递增的值:

```
iota(begin, end, start_value);
```

```
vector<int> v(5);
iota(v.begin(), v.end(), 0); // v = {0,1,2,3,4}
iota(v.begin(), v.end(), 1); // v = {1,2,3,4,5}

// 常见用法: 生成下标数组后按自定义key排序
vector<int> idx(n);
iota(idx.begin(), idx.end(), 0);
sort(idx.begin(), idx.end(), [&](int a, int b){
    return arr[a] < arr[b]; // 按arr值排序, idx存的是原下标
});
```

常见 String API

- 构造

```
string s;
string s(n, 'a'); // n个'a'
string s = "hello";
string s(s2, pos, len); // 从s2的pos开始取len个字符
```

- 增删改

```
s.push_back('c'); // 末尾加字符
s.pop_back(); // 末尾删字符
s.append(s2); // 末尾拼接
s += s2; // 末尾拼接 (更常用)
s.insert(i, s2); // 位置i插入字符串
```

```
s.insert(i, n, 'c');    // 位置i插入n个字符c
s.erase(i, len);       // 从i开始删len个字符
s.replace(i, len, s2);  // 从i开始len个字符替换为s2
s.clear();              // 清空
```

• 访问

```
s[i];                  // 下标
s.at(i);               // 下标 (越界抛异常)
s.front();             // 第一个字符
s.back();              // 最后一个字符
```

• 查找

```
s.find(s2);            // 第一次出现的位置, 找不到返回string::npos
s.find(s2, pos);       // 从pos开始找
s.rfind(s2);           // 最后一次出现的位置
s.find_first_of("abc"); // 第一个属于"abc"中任意字符的位置
s.find_last_of("abc");  // 最后一个属于"abc"中任意字符的位置
// 判断是否找到
if (s.find(s2) != string::npos) { ... }
```

• 子串与大小

```
s.substr(pos);         // 从pos到末尾
s.substr(pos, len);     // 从pos取len个字符
s.size();              // 长度 (同s.length())
s.empty();             // 是否为空
s.resize(n);           // 改变长度
```

• 转换 (需 <string>)

```
to_string(123);        // 数字转string
stoi("123");           // string转int
stoll("123");          // string转long long
stod("1.23");          // string转double
```

• 大小写 (需 <cctype>)

```
toupper('a');          // 转大写
tolower('A');          // 转小写
// 整个字符串转换:
for (auto& c : s) c = toupper(c);
```

- 字符判断 (需 `<cctype>`)

```
isdigit(c);    // 是否是数字
isalpha(c);    // 是否是字母
isalnum(c);    // 是否是字母或数字
isspace(c);    // 是否是空白字符
isupper(c);    // 是否大写
islower(c);    // 是否小写
```

- 流处理 (需 `<sstream>`)

```
// 按空格分割字符串
stringstream ss("hello world foo");
string token;
while (ss >> token) { cout << token << "\n"; }

// 数字和字符串互转
stringstream ss;
ss << 123;
string s = ss.str(); // "123"
```

getline

- 基本用法

```
string s;
getline(cin, s); // 读取一整行 (包含空格, 不包含换行符)
```

- 最常见的坑: cin 之后接 getline

```
int n;
cin >> n;
getline(cin, s); // ⚠ 会读到空行! 因为 cin >> n 留下了换行符

// 修复: 在 getline 之前加一句
cin >> n;
cin.ignore();    // 吃掉残留的换行符
getline(cin, s); // ✅ 正常读取
```

- 自定义分隔符

```
// 默认分隔符是 '\n', 可以改成其他字符
getline(cin, s, ','); // 读到逗号为止
getline(cin, s, ' '); // 读到空格为止
```

- 循环读取所有行

```
string line;
while (getline(cin, line)) {
    cout << line << "\n";
}
```

- 配合 stringstream 分割字符串

```
string line = "1,2,3,4";
stringstream ss(line);
string token;
while (getline(ss, token, ',')) {
    cout << token << "\n"; // 输出 1 2 3 4
}
```

unique

- $O(N)$ 去重 (只去相邻重复)
- 1. 离散化 (Sort + Unique + Erase) 必须先 Sort! $m = \text{unique}(a, a+n) - a$; 返回去重后末尾指针。

next_permutation $O(N)$

- 下一个全排列
- 暴力枚举全排列
- 生成字典序
- `do { ... } while(next_permutation(all(s)));`

lower_bound $O(\log N)$

- 找 $\geq \text{val}$ 的第一个位置
- LIS (最长上升子序列)
- 查找排名/前驱后继返回迭代器。
- `idx = lower_bound(all(v), val) - v.begin();`

__gcd

- $O(\log V)$
- 最大公约数

- 数论、化简分数
- $\text{lcm}(a,b) = a/\text{gcd}(a,b)*b$

fill

- $O(N)$
- 赋值
- 初始化数组/容器
- `fill(a, a+n, val);` 比 `memset` 安全, 支持任意值 (`memset` 只能 0/-1/0x3f)。

vector 我的最爱

- 支持可变大小, 不支持开出来的大小清空
- $O(1)$ `push_back()`, `pop_back()`
- $O(n)$ `insert()`, `erase()`
- 连续空间, 可以直接指针++
- `resize()` 改变大小且填充默认值; `reserve()` 只预留空间不改大小 (优化常数神器)。
- 邻接表存图 `vector<int> G[N]`

deque 支持双向

- 头尾插入放出 $O(1)$ `push_front`; `push_back`; `pop_front`; `pop_back`
- 适用于滑动窗口
- 0-1 BFS (边权只为0或1的最短路)

string

- 拼接 + 是 $O(N)$
- 配合 `stringstream` 做类型转换
- `s.find()` 找不到返回 `string::npos`。
- 千万别在循环里 `s = s + c`, 要用 `s += c`。

priority_queue 优先队列

- `push`, `pop`, $O(\log n)$ 方便贪心最大最小
- 可以通过重载运算符来达成最大或者这最小

list

- $O(1)$ 删除任意节点 `erase`
- 不能随机访问

stack

- 后缀表达式
- DFS

queue

- BFS

- **注意**队列不能遍历，没有operator[]，如果要遍历却不出队可以使用deque

set 自动去重的有序集合

- $O(\log n)$ 查找 `find()`;插入于删除
- **支持**`lower_bound(x)` `` `upper_bound(x)` 依旧 $O(\log n)$
- 场景：动态维护有序序列且快速去重;坐标压缩
- 清空数组`set.clear()`
- 注意：`bc.push_back({t, (int)cnt.size()})` (这里的cnt是set类型的)类似于这一句需要强制类型转换，因为`set.size()`函数返回的是无符号类型

multiset 和set完全一样但是允许重复元素

- 动态维护中位数 场景：滑动窗口中位数
- 贪心（类似堆，但支持删任意值）实现 $O(\log n)$ 美丽复杂度的查找与删除，利于维护优先队列（包括`lower_bound(x)` `` `upper_bound(x)`）
- **大坑：**`erase(val)` 会删掉所有等于 `val` 的元素；`erase(iter)` 只删一个。 `iter`是迭代器

map

(例子定义：`map<int,int> mp;`)

- 增删查均为 $O(\log N)$
- 离散化
- 自动排序的去重统计
- 启发式合并
- 支持 `mp[x]`快速访问与插入
- `mp.count(key)`判断成员是否存在（**仅仅判断请用这个**，如果用operator会插入对应键值对，，默认数值为0）
- 离散化映射；建图
- `mp`的遍历: `for (auto it = nums.begin(); it != nums.end(); it++)` 提取内容写法: `auto hsh=it->second;`
- 用map做索引放数组

```
map<int, vector<pii>> mp;
for (int i = 0; i < n; i++)
{
    mp[fdk(aa[i])].push_back({aa[i], i});
}
vi ans(n + 1);
for (auto &[k, hsh] : mp)
{
    vi pp;
    for (int i = 0; i < hsh.size(); i++)
    {
```

```

        pp.push_back(hsh[i].first);
    }
    sort(all(pp));

    for (int i = 0; i < hsh.size(); i++)
    {
        ans[hsh[i].second] = pp[i];
    }
}

```

std::unordered_set / unordered_map —— 哈希表（平均 $O(1)$ ）但是好像最坏能到 $O(n)$

我不会，也没做过类似的题，以下内容均由ai生成

- 平均时间复杂度 $O(1)$ ，find, insert, 等操作都快
- 无序
- 不支持lower_bound
- 键值唯一
- 哈希冲突严重时退化成链表 $O(n)$
- 字符串/大数 快速映射
- 频率统计

堆 std::priority_queue

- 好东西
- 最短路dijkstra
- 贪心
- 合并果子
- 写法:
- 在 priority_queue 的 cmp 中, return true 的含义是: “左边这个元素(a) 比 右边这个元素(b) 优先级更低。”
- top $O(1)$
- push/pop $O(\log N)$

```

struct Node {
    int val;
    int id;
};

// === 核心模板: 自定义比较结构体 ===
struct cmp {
    // 传入 const引用, 速度快且安全
    // 返回 true 表示: a 的优先级比 b "低" (a 应该沉在下面)
    bool operator()(const Node& a, const Node& b) {

        // --- 场景 1: 我是小根堆 (数值小的在堆顶) ---
        // 逻辑: 如果 a 的值 大于 b, 说明 a 比较“重”, a 应该沉下去
    }
}

```

```

        return a.val > b.val;

        // --- 场景 2: 我是大根堆 (数值大的在堆顶) ---
        // 逻辑: 如果 a 的值 小于 b, 说明 a 比较“弱”, a 应该沉下去
        // return a.val < b.val;

        // --- 场景 3: 多关键字排序 ---
        // 比如: 先按 val 小的排, val 一样按 id 大的排
        // if (a.val != b.val) return a.val > b.val; // val大的沉下去(小根)
        // return a.id < b.id; // id小的沉下去(大根)
    }
};
// === 核心模板:比较数字 ===

int dist[10005]; // 全局距离数组

// 堆里只存节点编号 int
struct cmp {
    bool operator()(int u, int v) {
        // 哪怕堆里只有 int, 我也可以引用外部数组来比较
        // 我要 distance 小的在堆顶 (Dijkstra 常见写法)
        return dist[u] > dist[v];
    }
};

// 使用
priority_queue<int, vector<int>, cmp> pq;

int main() {
    // 声明方式: <类型, 容器, 比较器>
    // 这里的 vector<Node> 是堆的底层容器, 必写, 但不用管它
    priority_queue<Node, vector<Node>, cmp> pq;

    pq.push({10, 1});
    pq.push({5, 2}); // 5 最小, 应该在最上面
    pq.push({20, 3});
    //用push是因为这样子两个都会变成node类型
    while(!pq.empty()) {
        Node top = pq.top();
        pq.pop();
        cout << "Val: " << top.val << endl; // 输出顺序: 5 -> 10 -> 20
    }
    return 0;
}

```

字典树模板

- Trie 的优势在于“快速查找前缀”或者“大量字符串排序”。

```
struct Trie
{
    int tre[MAXN][70];
    int cnt[MAXN];
    int idx = 0;

    void clear()
    {
        for (int i = 0; i <= idx; i++)
        {
            for (int j = 0; j < 65; j++)
                tre[i][j] = 0;
            cnt[i] = 0;
        }
        idx = 0;
    }

    int getnum(char ch)
    {
        if (ch >= 'A' && ch <= 'Z')
        {
            return ch - 'A';
        }
        else if (ch >= 'a' && ch <= 'z')
        {
            return ch - 'a' + 26;
        }
        else if (ch >= '0' && ch <= '9')
        {
            return ch - '0' + 52;
        }
    }

    void insert(string s)
    {
        int ptr1 = 0;
        int bra = 0;
        for (int i = 0; i < s.size(); i++)
        {
            if (!tre[ptr1][getnum(s[i])])
            {
                idx++;
                tre[ptr1][getnum(s[i])] = idx;
            }
            ptr1 = tre[ptr1][getnum(s[i])];
            cnt[ptr1]++;
        }
    }

    int query(string s)
    {
        int ptr2 = 0;
        for (int i = 0; i < s.size(); i++)
```

```
        {
            if (!tre[ptr2][getnum(s[i])])
            {
                return 0;
            }
            ptr2 = tre[ptr2][getnum(s[i])];
        }
        return cnt[ptr2];
    }
}myTrie;

void solve()
{
    myTrie.clear();
    int n, q;
    cin >> n >> q;
    for (int i = 0; i < n; i++)
    {
        string s;
        cin >> s;
        myTrie.insert(s);
    }
    for (int i = 0; i < q; i++)
    {
        string t;
        cin >> t;
        // 调用 query 并输出结果
        cout << myTrie.query(t) << "\n";
    }
}
```